AWS Certified Data Engineer – Associate (DEA-C01) — 15-Day Study Plan

A focused, hands-on 15-day plan to prepare for the **AWS Certified Data Engineer – Associate (DEA-C01) **exam. Each day =** read the concepts → build the project → drill questions → review**. By day 15 you'll have both the knowledge *and* a working data pipeline you built with your own hands.

15 days is aggressive but realistic **if** you already have basic programming or database experience and can study 3–4 hours/day. Brand new to SQL/Python? Use these 15 days for the foundation and add 1–2 weeks for mock exams and reinforcement.

The exam at a glance

Domain	Weight
Data ingestion and transformation	**34%**
Data store management	**26%**
Data operations and support	**22%**
Data security and governance	**18%**

- **65 questions** (50 scored + 15 unscored), **130 minutes**
- **Passing score: 720 / 1000**
- Format: multiple choice / multiple response
- Cost: ~$150 USD · Delivery: Pearson VUE (test center or online proctored)
- Official DEA-C01 exam guide

· In-scope services list

How to think about the exam: it rewards *service selection* and *architecture decisions* ("which service, why, and how do they connect?") far more than memorizing every feature flag. Almost every question is a short scenario — read for the constraint

(cost, latency, durability, least privilege, minimal ops) and pick the service that fits.

Daily routine (4-hour timetable)

Block	Time	What you do
🧠 **Concepts**	90 min	Read the day's topics + AWS docs/FAQ for each service
🛠️ **Hands-on**	90 min	Build that day's slice of the project in your AWS account
❓ **Questions**	45 min	Practice questions on the day's domain
📝 **Review**	15 min	Update your notes + flashcards ("when to use what")

Only have 2 hours? Keep the **hands-on (90 min)** + a short **15-min review** and

trim the reading. Doing beats reading for this exam.

⚠️ **Cost hygiene (do this every day):** set an **AWS Budgets alert** on day 1, prefer **serverless / on-demand** (Glue, Athena, Lambda, DynamoDB on-demand), stop/delete anything provisioned (Redshift clusters, EMR, RDS, MSK) at the end of each session, and run everything in **one region**. Athena, Glue, and small S3/DynamoDB usage cost cents.

15-day schedule with detailed concepts + daily project

Day 1 — Cloud & IAM foundations

Concepts to master

- **Global infra:** Regions, Availability Zones, edge locations; why multi-AZ = high availability and multi-region = disaster recovery.
- **Shared Responsibility Model:** AWS secures *of* the cloud; you secure *in* the cloud.
- **IAM:** users vs **roles** (roles = temporary credentials, preferred for services),

policies (identity vs resource-based), policy evaluation (explicit deny > allow),

least privilege, instance profiles, `sts:AssumeRole`.

- **Access & billing:** Console vs **CLI** vs SDK (boto3); **AWS Budgets** & cost alerts.
- **Exam angle:** "how should service X get permission to read S3?" → an **IAM role**, not access keys.

🛠️ **Hands-on:** Create the AWS account (or sub-account), set a **budget alert**,

configure the **AWS CLI**, and create a **least-privilege IAM role** a Glue job could use

(trust policy for `glue.amazonaws.com` + S3 read/write to one bucket).

Day 2 — SQL for data engineering

Concepts to master

- Core: `SELECT`, `WHERE`, `GROUP BY`, `HAVING`, all **join** types, `UNION`.
- Advanced: **CTEs** (`WITH`), subqueries, **window functions** (`ROW_NUMBER`, `RANK`, `LAG/LEAD`, running totals), **deduplication** with `ROW_NUMBER() … PARTITION BY`.
- Date/time functions, `CASE`, `COALESCE`, type casting.
- Physical/logical: **indexes**, **transactions** (ACID), **normalization vs denormalization** (and when each is right for analytics).
- **Exam angle:** Athena/Redshift use standard SQL; expect dedup, windowing, and "why denormalize for OLAP" reasoning.

🛠️ **Hands-on:** Do **20+ SQL exercises** (any SQL sandbox). Write a dedup query and a

window-function ranking — you'll reuse these patterns in Athena.

Day 3 — Python & data formats

Concepts to master

- Python: collections (list/dict/set), functions, exceptions, modules, comprehensions, logging, virtual envs; **boto3** basics (client vs resource, paginators).

- **File formats — the big exam topic:**
- **Row formats:** CSV/JSON (human-readable, no schema enforcement), **Avro** (row-based, great for streaming + schema evolution).
- **Columnar: Parquet**/ORC (compressed, column-pruned, best for analytics/Athena — usually the *right answer* for a curated zone).
- **Compression:** Snappy (fast, splittable-ish), Gzip (smaller, not splittable), Zstd; splittability matters for parallel processing.
- **Partitioning** (e.g. `year/month/day`) to prune scanned data → cheaper Athena.

🛠️ **Hands-on:** Write a Python script that reads messy CSV, cleans/dedupes it, and writes **partitioned Parquet** (pandas/pyarrow). This is your project's transform logic.

Day 4 — Data-engineering foundations

Concepts to master

- **ETL vs ELT**, **batch vs streaming**, **OLTP vs OLAP**.
- **Dimensional modeling:** facts vs dimensions, **star vs snowflake** schema, grain.
- **Slowly Changing Dimensions (SCD):** Type 1 (overwrite), **Type 2** (history rows with effective dates) — know the difference cold.
- **Data quality** dimensions (completeness, uniqueness, validity), **idempotency** (safe re-runs), **schema evolution**.
- **Exam angle:** pick the modeling/loading strategy for a described reporting need.

🛠️ **Hands-on:** Design a small **star schema** for sales data (fact_sales + dim_date, dim_customer, dim_product) on paper/diagram; note where SCD Type 2 applies.

Day 5 — Amazon S3 & the data lake

Concepts to master

- Buckets, keys/**prefixes**, **storage classes** (Standard → IA → One-Zone-IA → Glacier Instant/Flexible/Deep Archive) and when each fits.

- **Lifecycle policies**, **versioning**, **replication** (CRR/SRR).
- **Encryption:** SSE-S3, **SSE-KMS** (audited, key policies), SSE-C; enforce with bucket policy.
- **Performance:** prefix parallelism, **multipart upload**, S3 **event notifications** (→ Lambda/SQS/EventBridge), **partition design**.
- **Lake zones:** raw → processed/staged → curated.
- **Exam angle:** cost-optimize with lifecycle + storage class; secure with KMS; trigger pipelines with events.

🛠️ **Hands-on:** Create the lake bucket with **raw / processed / curated** prefixes, turn on versioning + SSE-KMS, add a **lifecycle rule** (e.g. raw → IA after 30 days). Upload your sample source data to `raw/`.

Day 6 — Databases & data stores

Concepts to master

- **RDS / Aurora** (relational OLTP; Multi-AZ = HA, read replicas = scale reads; Aurora Serverless v2).
- **DynamoDB** (NoSQL key-value): **partition key** design, **sort key**, **LSI/GSI**, **on-demand vs provisioned** capacity, DynamoDB **Streams**, TTL — avoid hot partitions.
- **Redshift** (columnar MPP warehouse): **distribution styles** (KEY/EVEN/ALL), **sort keys**, Redshift Spectrum **(query S3),** materialized views**, RA3 + managed storage, Redshift Serverless, `COPY`/`UNLOAD`.
- **OpenSearch** (search/log analytics), **ElastiCache/MemoryDB** (caching).
- **Exam angle:** RDS/Aurora vs DynamoDB vs Redshift selection is heavily tested.

🛠️ **Hands-on:** Build a **service-selection comparison table** (workload → store). Model one DynamoDB access pattern (choose PK/SK for a lookup).

Day 7 — AWS Glue & Amazon Athena

Concepts to master

- **Glue Data Catalog** (central metastore; Hive-compatible), **crawlers**, classifiers, databases/tables, partitions.
- **Glue ETL:** Spark & **Python Shell** jobs, **DynamicFrames** vs DataFrames, **job bookmarks **(incremental), triggers/workflows,** Glue Data Quality**, Glue Studio.
- **Athena:** serverless SQL over S3, **workgroups** (cost controls), partitioning & **partition projection**, CTAS, cost = **data scanned** (Parquet + partitions = cheap).
- **Exam angle:** catalog + Athena is the default serverless analytics answer.

🛠️ **Hands-on:** Run a **crawler** over `raw/`, query it in **Athena**, then a **Glue job** to clean → Parquet into `curated/`; crawl and query the curated table.

Day 8 — Batch transformation (Glue vs EMR, Spark)

Concepts to master

- **Glue vs EMR:** Glue = serverless, less ops; **EMR** = full control (Spark/Hive/Presto/HBase), EMR on EC2 vs **EMR Serverless** vs EMR on EKS, spot for cost.
- **Spark fundamentals:** driver/executors, transformations vs actions, lazy evaluation, **shuffle**, **data skew**, **partitioning/repartition/coalesce**, broadcast joins, caching, `spark.sql.shuffle.partitions`.
- Cost/perf: right-size DPUs/workers, avoid small files, push down filters.
- **Exam angle:** diagnose a slow/expensive Spark job (skew, too many small files, wrong join).

🛠️ **Hands-on:** Add a **join + aggregation** to your Glue Spark job; identify one inefficiency (small files or skew) and fix it (repartition / compact output).

Day 9 — Streaming ingestion

Concepts to master

- **Kinesis Data Streams** (real-time, **shards**, retention, consumers/**KCL**, enhanced fan-out, resharding).
- **Kinesis Data Firehose** (near-real-time **delivery** to S3/Redshift/OpenSearch, buffering, format conversion to Parquet, no code) — **Streams vs Firehose** is a classic question.
- **Amazon MSK / Kafka** (topics, **partitions**, offsets, consumer groups, MSK Serverless) vs Kinesis.
- Delivery guarantees: at-least-once vs exactly-once, ordering, **checkpointing**.
- Managed **Flink** (Kinesis Data Analytics) for stream processing.
- **Exam angle:** latency + transformation + destination → pick Streams vs Firehose vs MSK.

🛠️ **Hands-on:** Design (or build) a streaming path: producer → **Firehose** → `raw/` in S3 (Parquet). Diagram how it also fans out to Redshift.

Day 10 — Pipeline orchestration

Concepts to master

- **Step Functions** (state machine, retries/**catch**, Map/Parallel, Standard vs Express) — the AWS-native orchestrator.
- **Amazon MWAA** (managed **Airflow**/DAGs) — for complex, code-first orchestration; **Step Functions vs MWAA** trade-off.
- **EventBridge** (event bus, rules, schedules), **Lambda** (glue code, 15-min limit), **SQS** (queue, decoupling, **DLQ**) vs **SNS** (pub/sub fan-out).
- Patterns: **retries with backoff**, **dead-letter queues**, idempotency, dependency ordering.
- **Exam angle:** choose the orchestration/eventing service for the described workflow.

🛠️ **Hands-on:** Build a **Step Functions** workflow that runs your crawler → Glue job → (on failure) sends SNS/DLQ. Even a detailed diagram counts if cost is a concern.

Day 11 — Migration & integration

Concepts to master

- **AWS DMS** (homogeneous & heterogeneous migration, **full load + CDC**, replication instances, task settings), **Schema Conversion Tool (SCT)**.
- **DataSync** (large file/object transfer to S3/EFS/FSx), **Storage Gateway** (hybrid), **Transfer Family** (SFTP → S3).
- **DMS vs DataSync:** databases/CDC vs files/objects.
- Migration validation, minimal-downtime cutover (full load then CDC catch-up).
- **Exam angle:** on-prem DB → AWS with minimal downtime → **DMS full load + CDC**.

🛠️ **Hands-on:** Design an on-prem-DB-to-AWS migration (DMS + SCT) with a minimal-downtime cutover plan; write it up as a short runbook.

Day 12 — Operations, monitoring & IaC

Concepts to master

- **CloudWatch:** metrics, **logs**, **alarms**, dashboards, Logs Insights, custom metrics; **CloudTrail** (API audit) — **CloudWatch vs CloudTrail** (performance vs who did what).
- Service logs: Glue/EMR/Lambda logs, Redshift/Athena query monitoring.
- Reliability: failure recovery, retries, **quotas/limits**, backfills.
- **IaC:** CloudFormation / **CDK** / Terraform for repeatable pipelines.
- **Exam angle:** how to detect, alarm on, and recover from a failed pipeline.

🛠️ **Hands-on:** Add **CloudWatch alarms** (Glue job failure, DLQ depth) and write a **troubleshooting checklist** for a failed pipeline run.

Day 13 — Security & governance

Concepts to master

- **IAM** deep dive (identity vs resource policies, conditions), **KMS** (CMKs, key

policies, envelope encryption, rotation), **Secrets Manager** vs SSM Parameter Store.

- **Lake Formation:** central lake permissions, **row/column-level** & tag-based access, LF-tags, blueprints; how it layers over the Glue Catalog.
- Encryption **in transit** (TLS) & **at rest** (SSE-KMS everywhere).
- **Macie** (PII discovery), data **masking/redaction**, **audit logging**, retention.
- **Exam angle:** least-privilege + fine-grained lake access + encryption defaults.

🛠️ **Hands-on:** Secure the lake: KMS on all zones, scoped IAM roles, and apply

Lake Formation column-level permissions to one table (concept + setup).

Day 14 — Full mock exam & repair

- Take **one timed 65-question mock** under real conditions.
- Review **every** wrong *and* guessed answer; note the underlying concept, not just the right letter.
- Rebuild weak areas; expand your **"when to use what"** cheat sheet.
- **Target 75–80%+** before you schedule the real exam.

🛠️ **Hands-on:** Fix any project gap the exam exposed (e.g. add a data-quality check or a

missing partition).

Day 15 — Final review

- Take a **second timed mock**; confirm you're at/above target.
- Review architecture patterns, security defaults, failure handling, cost optimization, and all **service comparisons** (below).
- **Don't learn big new topics today** — consolidate with flashcards and light review.
- Log a good night's sleep before exam day. ✅

The one project you build across 15 days

Everything connects into a single **serverless lakehouse**:

```
CSV/JSON source
```

```
      ↓
Amazon S3  raw zone            ← Day 5
      ↓
Glue Crawler → Data Catalog    ← Day 7
      ↓
Glue ETL (Spark): clean, dedup, join, → Parquet   ← Days 7–8
      ↓
Amazon S3  curated zone
      ↓
Athena  (and/or Redshift Spectrum)    ← Days 6–7
      ↓
Step Functions orchestration + CloudWatch alarms   ← Days 10, 12
```

Grow it with these as you go:

- ⏱️ Partition by `year/month/day` (Day 3/5)
- 🔐 KMS encryption on every zone (Day 5/13)
- 👤 Scoped IAM execution roles (Day 1/13)
- 🔁 Retry + DLQ + failure handling (Day 10)
- ✅ A data-quality check (Day 7/14)
- 🗄️ Lifecycle policies (Day 5)
- 🌊 A written design for how it changes for **streaming** (Day 9)
- 🏛️ Lake Formation column-level permissions (Day 13)

High-priority service comparisons (know these cold)

Comparison	One-line decision rule
Glue vs EMR	Serverless, less ops (Glue) vs full control over big Spark/Hadoop clusters (EMR).
Athena vs Redshift	Ad-hoc SQL over S3, pay-per-scan (Athena) vs fast repeated BI on loaded data, MPP warehouse (Redshift).
RDS/Aurora vs DynamoDB	Relational/OLTP, joins, SQL (RDS/Aurora) vs key-value, huge scale, single-digit-ms (DynamoDB).
Kinesis Streams vs Firehose vs MSK	Custom real-time consumers (Streams) vs no-code delivery to S3/Redshift (Firehose) vs Kafka ecosystem/portability (MSK).

Step Functions vs MWAA	Native, low-ops state machine (Step Functions) vs complex code-first Airflow DAGs (MWAA).
SQS vs SNS vs EventBridge	Point-to-point queue (SQS) vs pub/sub fan-out (SNS) vs event routing + filtering + schedules (EventBridge).
Glue Catalog vs Lake Formation	Metadata store (Catalog) vs fine-grained permissions on that metadata (Lake Formation).
CloudWatch vs CloudTrail	Metrics/logs/alarms — performance (CloudWatch) vs API audit — who did what (CloudTrail).
DMS vs DataSync	Database migration + CDC (DMS) vs file/object transfer (DataSync).
CSV/JSON vs Parquet/Avro	Human-readable/interchange (CSV/JSON) vs columnar-analytics (Parquet) / row-streaming + schema evolution (Avro).
Provisioned vs serverless	Predictable heavy load, lowest unit cost (provisioned) vs spiky/unknown load, no capacity mgmt (serverless).

Resources

- **Official:** DEA-C01 exam guide

· In-scope services

· AWS Skill Builder (official practice question set)

- **Read the FAQ page** of each core service (S3, Glue, Athena, Redshift, Kinesis, DynamoDB, Step Functions, Lake Formation) — FAQs mirror how the exam phrases things.

- **Whitepapers:** *AWS Well-Architected — Analytics Lens*, *Big Data Analytics Options*.
- Practice exams from a reputable provider for Days 14–15.

Realistic expectations

- **Strong SQL/Python/AWS background:** 15 days at 3–4 hrs/day is doable.
- **Newer to the stack:** use these 15 days for foundations, then add **1–2 weeks** of mock exams + hands-on before booking.

- The build-it-yourself project is what makes the concepts stick — **don't skip the daily hands-on**. Reading alone rarely passes this exam.

Educational material — verify details against current AWS documentation before relying on them. Good luck! 🚀